RĪGAS TEHNISKĀ UNIVERSITĀTE

Datorzinātnes, informācijas tehnoloģijas un enerģētikas fakultāte

Atskaite par pirmo praktisko darbu

Studiju kurss “Mākslīgā intelekta pamati”

Komandas numurs: 49
Darba izpildītāji: Iaroslav Viazmitin, 241RDB360
Nikita Karpovs, 241RDB332
Matīss Vasks, 231RDB154
Līna Skripko, 231RDB169
Deivids Kārkliņš, 231RDB183

Mācībspēks: Alla Anohina-Naumeca

Saite uz projektu/kodu:
https://github.com/ImWeffi/mip-1

2025./2026.studiju gads

Mākslīgā intelekta rīku izmantošanas paziņojums

Rīks (nosaukums, gads, versija)	Rīka saite	Rīka izmantošanas nolūks (atskaites teksta ģenerēšana, spēles pamatalgoritmu un datu struktūru izveide, spēles lietotāja saskarnes izveide, ideju ģenerēšana vai ietiekumu saņemšana par heiristiskā novērtējuma funkciju, kļūdu labošana izveidotajā kodā, papildu skaidrojumu iegūšana, eksperimentu plānošana, u.c.)
ChatGPT, 2026, 5.3	https://chatgpt.com	spēles pamatalgoritms un datu strukūra izveide
ChatGPT, 2026, 5.3	https://chatgpt.com	saskarsnes pievienošāna ar spēles pamatalgoritmu.
Gemini, 2026, Pro 3.1	Google Gemini	Atskaites teksta ģenerēšana, ideju ģenerēšana un ieteikumu saņemšana

Programmatūras darbības demonstrācijas piemērs

Papildu prasības programmatūrai:

Spēles sākumā cilvēks-spēlētājs norāda spēlē izmantojamas skaitļu virknes garumu, kas var būt diapazonā no 15 līdz 20 skaitļiem. Spēles programmatūra gadījuma ceļā saģenerē skaitļu virkni atbilstoši uzdotajam garumam, tajā iekļaujot skaitļus 1, 2, 3 un 4.

Spēles apraksts:

Spēles sākumā ir dota ģenerētā skaitļu virkne. Katram spēlētājam ir 0 punktu. Spēlētāji izpilda gājienus secīgi. Gājiena laikā spēlētājs:

- var paņemt jebkuru skaitli no virknes un pieskaitīt to savam punktu skaitam;
- sadalīt «2» divos skaitļos «1» un «1», bet par to neiegūt punktus;
- sadalīt “4” divos skaitļos “2” un “2” un par to pieskaitīt savam punktu skaitam 1 punktu.

Spēle beidzas, kad virkne paliek tukša. Uzvar spēlētājs, kam ir vairāk punktu. Ja punktu skaits ir vienāds, tad rezultāts ir neizšķirts.

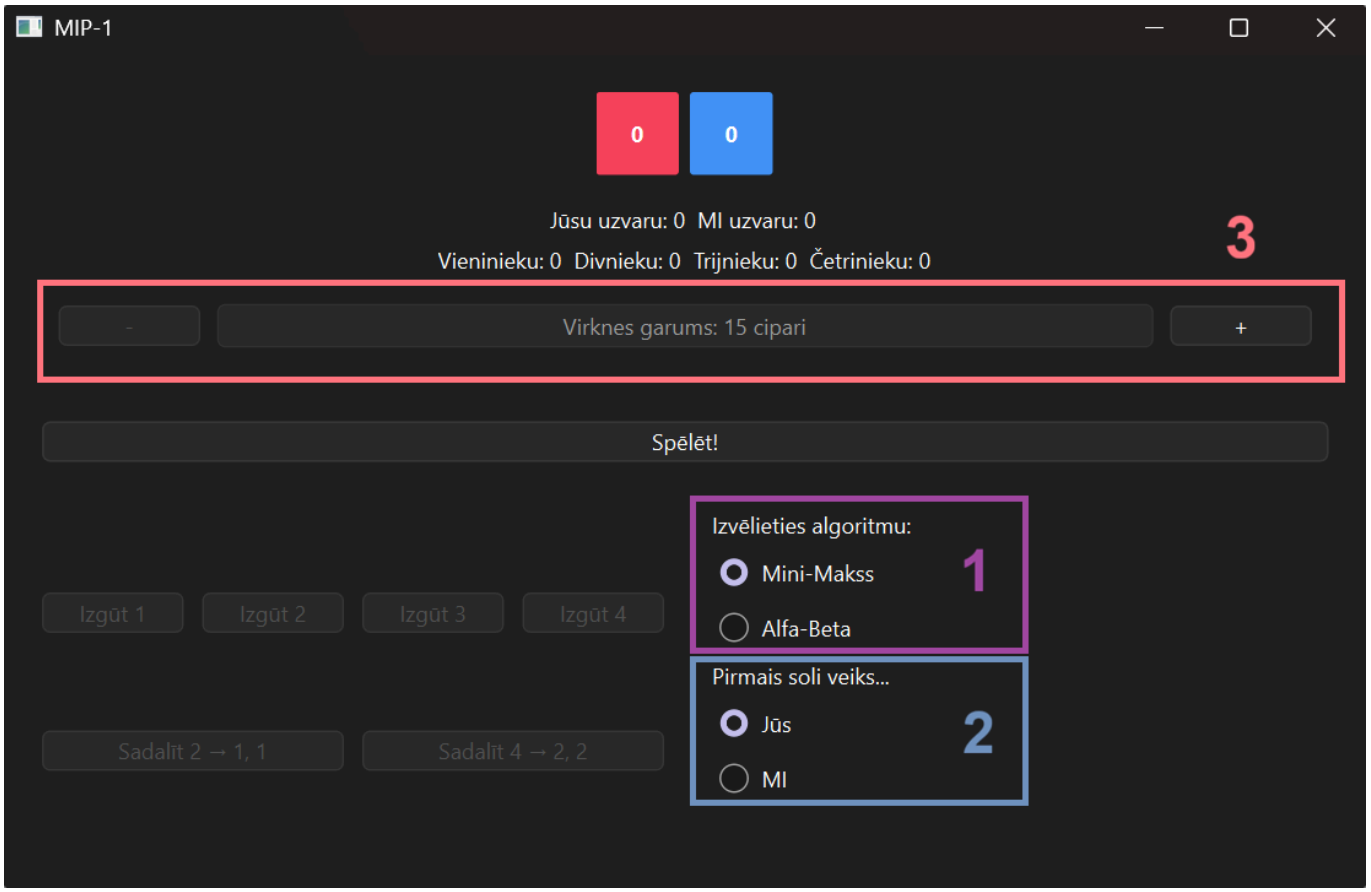
Spēles darbības demonstrācijas piemērs ar ekrānuzņēmumiem un skaidrojumiem:

1. Programmatūras palaišana & konfigurācija

Kad programmatūra tiek palaista lietotājs no paša sākuma var iepazīties ar spēles iestatīšanas iespējām:

1. Algoritma izvēle – ("Minimakss" vai "Alfa-Beta"), ietekmēs datora gājienu veikšanas stratēģiju.
2. Gājienu priekšroka – (cilvēks vai dators), nosaka, kurš veic pirmo gājienu pēc virknes ģenerācijas.
3. Virknes garums – (no 15 līdz 20 cipariem), nosaka sākotnējo skaitļu virknes garumu.

   Spēlētājs drīkst mainīt šos 3 spēles iestatījumus pēc savas vēlmes vai sākt spēli pēc noklusējuma iestatījumiem.

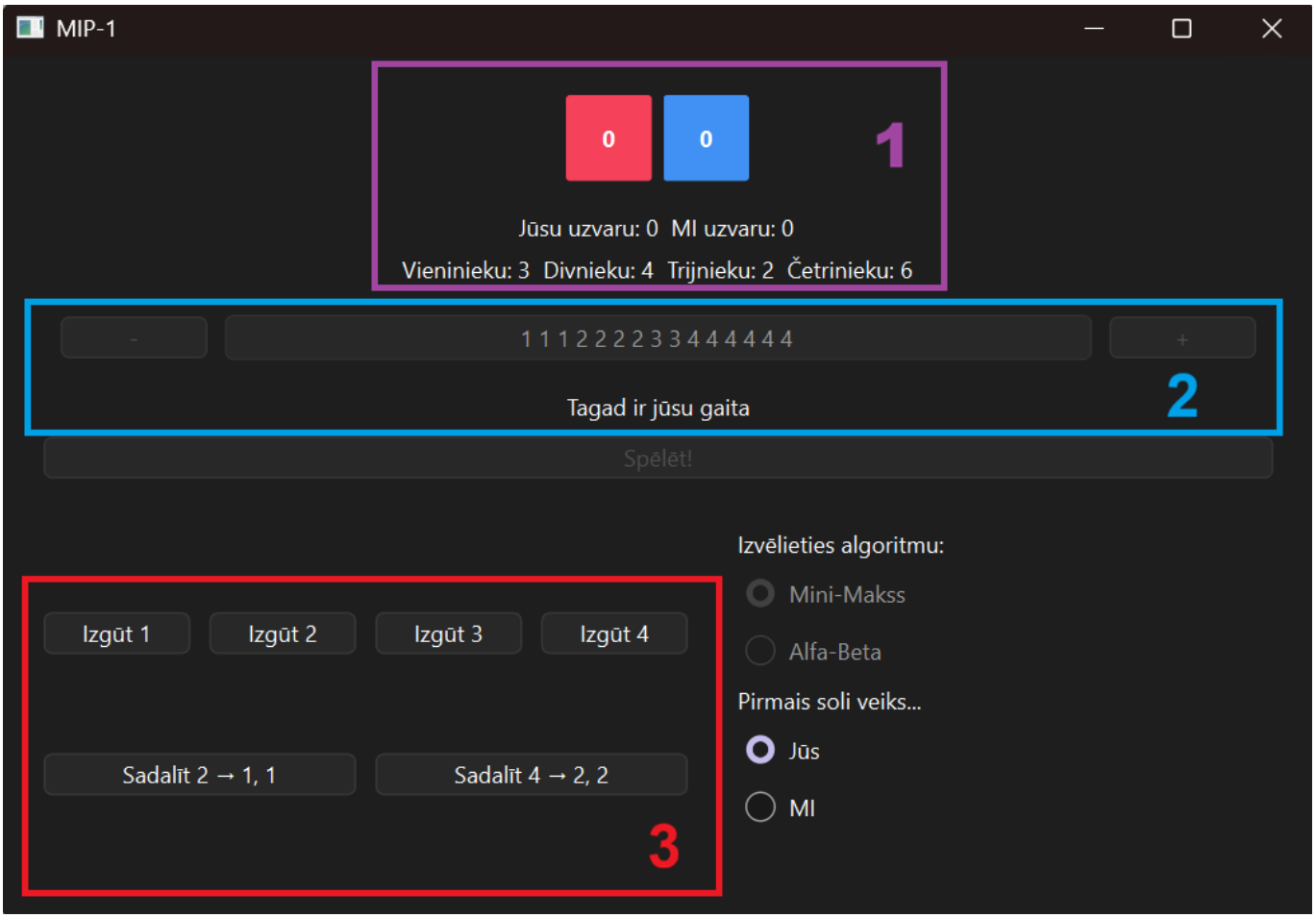


2. Spēles sākums & gaita

Pēc parametru iestatīšanas var sākt spēli. Pēc tās uzsākšanas lietotājam tiks parādīti atjaunināti spēles loga elementi, kuros atrodas nepieciešamā informācija:

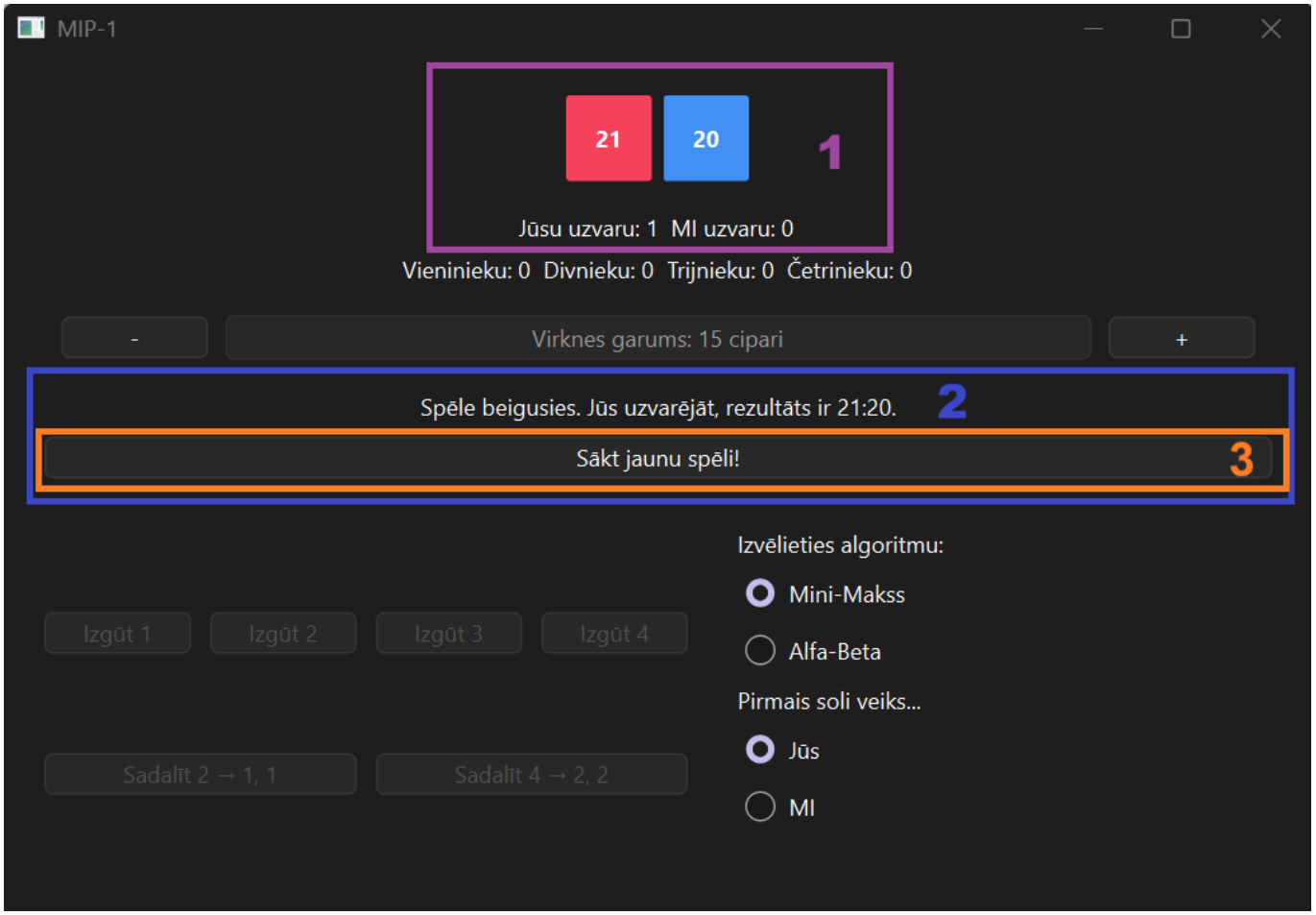
1. Spēļu statistika:
   1.1. Pašreizējās spēles punktu skaits spēlētājiem – Cilvēkam un Datoram
   1.2. Programmas darbības laikā reģistrētais līdz šim ik spēlētāju uzvaru skaits
   1.3. Ciparu atkārtošanu skaits (vieglākai pārredzēšanai)
2. Skaitļu virkne novietota izvēlētā virknes garuma vietā, kas attēlo pašreizējo spēles atlikušo ciparu skaitu, kā arī paziņojums zem tās par tekošo gaitas veicēju.
3. Iespējamo gājienu izvēle.

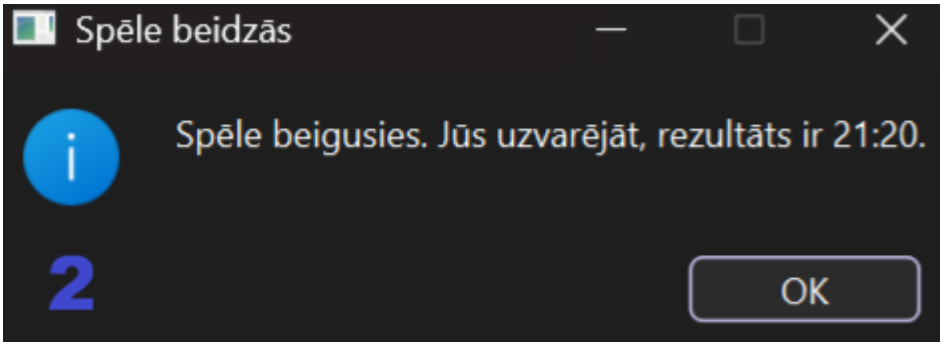
Spēles gaita turpinās attiecīgi pēc dotā varianta spēles apraksta.



3. Spēles beigas

Spēle beidzas kad tiek pabeigts pēdējais iespējamais gājiens un tiek izvadīts logs ar virsrakstu "Spēle beidzās" un tiek paziņots rezultāts. Uzvaru skaits tiek saglabāts un notikušās spēles rezultāts tiek parādīts - Cilvēks Dators. Lietotājs tālāk var izvēlēties turpināt spēli vai pārkonfigurēt spēles procesa nosacījumus.





Datu struktūru un algoritmu apraksts

Datu struktūru apraksts:

1. Šīs programmas pamatā ir spēles koks, izmantojot **TreeNode** klasi. Katrs koka mezgls glabā konkrētu spēles situāciju **(SolverState)** un gājienu, kā līdz tai nonāca. Tāpat katram mezglam ir saraksts ar bērniem, visiem iespējamajiem nākamajiem gājieniem. Tieši šāda struktūra ļauj Minimax un Alfa-beta algoritmiem iziet cauri variantiem un atrast pašu izdevīgāko gājienu.

2. Pati spēles informācija tiek fiksēta **SolverState** objektā, kur datu glabāšanai izmantoti kortēži **(tuple)**.Skaitļu virkne un spēlētāju punkti tiek glabāti kā kortēži.

3. Saraksti **(list)** tiek izmantoti priekš datu modificēšanai. Piemēram, ģenerējot jaunu spēles situāciju, sākotnējais kortēžs tiek konvertēts par sarakstu, lai tajā varētu veikt aprēķinus, un pēc tam pārvērsts atpakaļ kortēžā.

4. Lai dažādas programmas daļas darbotos kopa, bet paliktu neatkarīgas, tiek izmantotas **(dict)**. Tās glabā atsauces uz funkcijām, kas jāizsauc pie noteiktām izmaiņām, piemēram, kad spēlētājs veic gājienu vai tiek mainīti algoritma iestatījumi.

5. Tiek izmantots datu klase (@dataclass), kurš tiek izmantos: **Action, SolverState, TreeNode, SearchStats**. Tas ļauj viegli nodalīt pašu spēles loģiku no tehniskajiem datiem un aprēķinu gaitas.

Heiristiskā novērtējuma funkcijas apraksts:

```
def heuristic(self, state: SolverState, maximizing_player: int) -> float:
    my_score = state.scores[maximizing_player]
    opp_score = state.scores[1 - maximizing_player]
    score_diff = my_score - opp_score

    seq = state.sequence
    count1 = seq.count(1)
    count2 = seq.count(2)
    count3 = seq.count(3)
    count4 = seq.count(4)

    material = count1 * 1.0 + count2 * 2.0 + count3 * 3.0 + count4 * 4.0
    split_potential = count4 * 0.6 + count2 * 0.2
    turn_bonus = 0.3 if state.current_player == maximizing_player else -0.3
    return score_diff * 10 + material * 0.15 + split_potential + turn_bonus
```

Heiristiskā funkcija ir izveidota, lai dators spētu novērtēt nenoterminalizētus spēles stāvokļus gadījumos, kas sasniegts maksimālais meklēšanas dziļums - *max_depth* . Šāda funkcija tika izveidota, lai sabalansētu "mantkārīgu" tiešo punktu gūšanu ar tālredzīgu spēles laukuma kontroli, īpaši izceļot spēles unikālo mehāniku - ciparu dalīšanu.

1. **Rezultātu starpība** *score_diff * 10* Šis ir pats svarīgākais faktors tas ir MI rezultāta un pretinieka rezultāta starpība. Tiek reizināts ar lielu svaru (10), lai algoritms primāri fokusētos uz reālo punktu gūšanu vai pretinieka punktu atņemšanu, nevis tikai "potenciālu" krāšanu.

2. **Materiāla vērtība** *material * 0.15* Novērtē palikušo ciparu kopējo summu, katrs cipars reizināts ar tā skaitlisko vērtību. Tas ir palīdz algoritmam saprast, cik daudz resursu vēl ir atlicis spēlē, un nedaudz preferēt stāvokļūs, kur spēlē ir vairāk vērtīgo ciparu, ko potenciāli iegūt vēlāk.

3. **Sadalīšanas potenciāls** *split_potential = count4 * 0.6 + count2 * 0.2* Tā kā 4 sadalīšana dod 1 punktu un divus 2 (kuri katrs dod 2 punktus vai iespēju dalīt tālāk), četriniekiem ir augsts stratēģiskais potenciāls (svars 0.6). Divnieku sadalīšana nedod tiešus punktus, bet dod iespēju mainīt gājienu secību un paildzināt spēli, tāpēc tiem dots mazāks, bet tomēr pozitīvs svars (0.2).
4. **Gājiena bonuss** *turn_bonus = 0.3 vai -0.3* Tiek piešķirts neliels bonuss (+0.3), ja novērtējamajā stāvoklī gājiens pieder MI spēlētājam (iniciatīva dod priekšrocību), un sods (-0.3), ja gājiens ir pretiniekam.

Algoritmu apraksts:

1. **Spēles stāvokļa apraksts**. Spēles stāvoklis tiek glabāts struktūrā SolverState. Tas stur pašreizējo skaitļu virkni, abu spēlētāju punktus un informāciju par to, kurš spēlētājs pašlaik izdara gājienu.
2. **Spēles koka ģenerāšana**. Spēļu koks tiek veidots, definējot visas iespējamās darbības noteiktā stāvoklī un aprēķinot, kāds būs nākamais stāvoklis pēc katras darbības veikšanas.
   2.1 **Iespējamo gājienu iegūšana.** Metode get_legal_actions pārskata visus skaitļus virknes elementu sarakstā. Katram skaitlim vienmēr ir atļauta darbība ‘’take’’. Ja elements ir 2, iespējama darbība ‘’split2’’. Ja elements ir 4, iespējama darbība ‘’split4’’.
   2.2 **Nākamā stāvokļa iegūšana.** Metode apply_action realizē pāreju no viena spēles stāvokļa uz nākamo. Darbības ‘’take’’ gadījumā izvētētais skaitlis tiek noņemts no virknes un pieskaitīts pašreizējā splēletāja punktiem. ‘’Split2’’ gadījumā skaitlis 2 tiek aizvietots ar diviem vieniniekiem. ‘’Split4’’ gadījumā skaitlis 4 tiek aizvietots ar diviem divniekiem un spēlētājs iegūst 1 punktu.

3. **Heiristisko vērtējumu piešķiršana virsotnēm.** Ja tiek sasniegts termināls stāvoklis, tiek izmantota precīza novērtēšanas funkcija evaluate_terminal, kas apreķina punktu starpību starp maksimizējošo spēlētāju un pretnieku. Ja koks dzīļuma ierobežojuma dēļ termināls stāvoklis vēl nav sasniegts, tiek izmantota heiristiskā novērtēšanas funkcija ‘’heuristic’’. Tā ņem vērā punktu starpību, virknes sastāvu, iespējamo nākotnes ieguvumu no sadalīšanas (split_potencial) ub nelielu bonusu tam spēlētājam, kuram pašlaik ir gājiens. Šī funkcija piešķir katrai nebeigu virsotnei skaitlisku vērtējumu, kas raksturo, cik labvēlīgs konkrētais stāvoklis ir datoram.
4. **Gājienu sakārtošana**. Lai uzlabotu algoritma efektivitāti, pirms koka pārmeklēšanas gājieni tiek sakārtoti ar metodi ‘’order_actions’’. Augstāka prioritāte tiek dotā darbībai ‘’take’’ ar lielāku skaitli, darbībai ‘’split4’’ un darbībai ‘’split2’’. Šāda sakārtošana ir īpaši noderīga Alfa-beta algoritmam, jo ļauj agrāk sasniegt atzarošanas gadījumus.
5. **Spēles algoritma pielietojums.** Projektā ir realizēti divi klasiskie spēļu teorijas algoritmi. **Minimakss** un **Alfa-beta atzarošana**.
   5.1. Minimakss algoritms. Minimakss rekursīvi pārmeklē spēles koku līdz terminālam stāvoklim vai līdz norādītajam dziļumam. Ja gājiens ir maksimizējošā spēlētāja kārta, tiek izvēlēta darbība ar lielāko novērtējumu. Ja gājiens ir minimizējošā spēlētāja kārta, tiek izvēlēta darbība ar mazāko novērtējumu. Minimakss algoritms nodrošina racionālu gājiena izvēli, pieņemot, ka arī pretinieks spēlē optimāli.

```
def minimax(
        self,
        state: SolverState,
        depth: int,
        maximizing_player: int,
        node: Optional[TreeNode] = None
    ) -> tuple[float, Optional[Action]]:
        if node is None:
            node = TreeNode(state=state, depth=depth)
            self.stats.generated_nodes += 1
            self.last_tree_root = node

        if state.is_terminal():
            self.stats.evaluated_nodes += 1
            value = self.evaluate_terminal(state, maximizing_player)
```

```
        node.value = value
        return value, None

    if depth == 0:
        self.stats.evaluated_nodes += 1
        value = self.heuristic(state, maximizing_player)
        node.value = value
        return value, None

    actions = self.order_actions(state, self.get_legal_actions(state))
    is_max_turn = state.current_player == maximizing_player
    best_action: Optional[Action] = None

    if is_max_turn:
        best_value = -math.inf

        for action in actions:
            next_state = self.apply_action(state, action)
            child = TreeNode(state=next_state, action=action, depth=depth - 1)
            self.stats.generated_nodes += 1
            node.children.append(child)

            value, _ = self.minimax(
                next_state,
                depth - 1,
                maximizing_player,
                child
            )

            if value > best_value:
                best_value = value
                best_action = action

        node.value = best_value
        return best_value, best_action

    best_value = math.inf

    for action in actions:
        next_state = self.apply_action(state, action)
        child = TreeNode(state=next_state, action=action, depth=depth - 1)
        self.stats.generated_nodes += 1
        node.children.append(child)

        value, _ = self.minimax(
            next_state,
            depth - 1,
            maximizing_player,
            child
        )

        if value < best_value:
            best_value = value
            best_action = action
```

```
        node.value = best_value
        return best_value, best_action
```

5.2. **Alfa-beta algoritms.** Alfa-beta algoritms darbojas līdzīgi Minimax, taču papildus izmanto robežvērtības alpha un beta, lai pārtrauktu to zaru izskatīšanu, kuri nevar ietekmēt gala rezultātu.

```
def alphabeta(
        self,
        state: SolverState,
        depth: int,
        alpha: float,
        beta: float,
        maximizing_player: int,
        node: Optional[TreeNode] = None
    ) -> tuple[float, Optional[Action]]:
        if node is None:
            node = TreeNode(state=state, depth=depth)
            self.stats.generated_nodes += 1
            self.last_tree_root = node

        if state.is_terminal():
            self.stats.evaluated_nodes += 1
            value = self.evaluate_terminal(state, maximizing_player)
            node.value = value
            return value, None

        if depth == 0:
            self.stats.evaluated_nodes += 1
            value = self.heuristic(state, maximizing_player)
            node.value = value
            return value, None

        actions = self.order_actions(state, self.get_legal_actions(state))
        is_max_turn = state.current_player == maximizing_player
        best_action: Optional[Action] = None

        if is_max_turn:
            value = -math.inf

            for action in actions:
                next_state = self.apply_action(state, action)
                child = TreeNode(state=next_state, action=action, depth=depth - 1)
                self.stats.generated_nodes += 1
                node.children.append(child)

                child_value, _ = self.alphabeta(
                    next_state,
                    depth - 1,
                    alpha,
                    beta,
                    maximizing_player,
                    child
                )
```

```
            if child_value > value:
                value = child_value
                best_action = action

            alpha = max(alpha, value)
            if beta <= alpha:
                break

        node.value = value
        return value, best_action

    value = math.inf

    for action in actions:
        next_state = self.apply_action(state, action)
        child = TreeNode(state=next_state, action=action, depth=depth - 1)
        self.stats.generated_nodes += 1
        node.children.append(child)

        child_value, _ = self.alphabeta(
            next_state,
            depth - 1,
            alpha,
            beta,
            maximizing_player,
            child
        )

        if child_value < value:
            value = child_value
            best_action = action

        beta = min(beta, value)
        if beta <= alpha:
            break

    node.value = value
    return value, best_action
```

6. **Uzvaru nesošo ceļu atrašana.** Šajā darbā optimālais ceļš tiek noteikts, atkārtoti pielietojot izvēlēto spēles algoritmu (Minimakss vai Alfa-Beta) katram spēles stāvoklim, lai izvēlētos labāko nākamo gājienu. Funkcija *best_move* nosaka, kuru algoritmu izmantot, un atgriež optimālo gājienu. Praktiski tas nozīmē, ka dators katrā savā gājienā ģenerē iespējamos nākamos stāvokļus, piešķir tiem novertējumu ar Minimakss vai Alfa-beta algoritmu, izvēlas darbību, kas ved uz visizdevīgāko stāvokli. Tādējādi uzvaru nesošais ceļš netiek glabāts kā atsevišķs saraksts, bet tiek konstruēts pakāpeniski, katrā solī izvēloties labāko nākamo virzienu spēles kokā.

Algoritmu salīdzinājums:

Eksperimentā tika aizvadītas 20 spēles - 10 reizes ar katru algoritmu (Minimakss un Alfa-Beta). Spēles gājieni tika veikti pārmaiņus: pirmo gājienu pildīja cilvēks, otro dators, trešo cilvēks, ceturto dators utt.

5 spēlēs tika izvēlētas labākas vērtības un otrās 5 tika izvēlētes gan otrais labākais solis, gan skaitļa sadalīšana un tas viss tika uzskaitīts kurā spēlē tas tika veikts, lai salīdzinātu datora rezultātu. Dati tika apkopoti tabulā kas ir apakšā.

Apzīmējumi:
(C) - Cilvēks iet pirmais
(D) - Dators iet pirmais

(C), (D) - Šajos gājienos tika optimālākais gājiens (labākais).
(C), (D) - Šajos gājienos tika izvēlētas visādas vērtības, otrās labākās vērtības vai skaitļu sadalīšana.

Eksperimenta rezultāti:

Minimakss algoritms

Spēles Nr.	Izvēlētais virknes garums	Cilvēka punktu skaits	Datora punktu skaits	Datora novērtēto virsotņu skaits	Datora vidējais laiks gājiena izpildei	Kopējais uzvaru skaits cilvēkam	Kopējais uzvaru skaits datoram
1 (C)	20	25	23	807847	0.535235s	1	0
2 (D)	20	25	26	2287258	1.546542s	1	1
3 (C)	19	26	24	1110002	0.821068s	2	1
4 (D)	19	19	21	785039	0.520537s	2	2
5 (C)	18	21	20	406223	0.292594s	3	2
6 (D)	18	21	30	1219226	0.738281s	3	3
7 (C)	17	23	30	1155650	0.632969s	3	4
8 (D)	17	20	30	3262133	1.668992s	3	5
9 (C)	16	15	20	475411	0.276953s	3	6
10 (D)	16	13	24	267514	0.152753s	3	7

Alfa-Beta algoritms

Spēles Nr.	Izvēlētais virknes garums	Cilvēka punktu skaits	Datora punktu skaits	Datora novērtēto virsotņu skaits	Datora vidējais laiks gājiena izpildei	Kopējais uzvaru skaits cilvēkam	Kopējais uzvaru skaits datoram
1 (C)	20	28	28	4670	0.012909s	0	0
2 (D)	20	22	24	5841	0.014920s	0	1
3 (C)	19	25	24	3678	0.010403s	1	1
4 (D)	19	24	25	4301	0.011435s	1	2
5 (C)	18	20	18	2039	0.005884s	2	2
6 (D)	18	25	27	3796	0.010896s	2	3
7 (C)	17	25	26	3454	0.007350s	2	4
8 (D)	17	19	28	3379	0.008430s	2	5
9 (C)	16	16	17	2240	0.006577s	2	6
10 (D)	16	14	22	2476	0.005955s	2	7

- **Minimakss** algoritms izstaigā pilnīgi visus iespējamos koka zarus līdz noteiktajam dziļumam. Tas ir drošs, taču ļoti lēns un resursietilpīgs, jo pārbauda arī tos gājienus, kas acīmredzami ir ļoti slikti. Tas arī pierādās tajā, ka sākotnēji Minimakss ar *depth = 6*, mēs saskārāmies ar ļoti ilgu datora skaitļošanu pie Minimakss algoritma izvēles, jo katrā gajienā gāja pilnībā visiem virknes skaitļiem cauri tas datoram aizņēma vairākas minūtes un kā rezultātā spēle apstājās līdz dators nebija pabeidzis skaitļošanu.

- **Alfa-Beta** ir principā uzlabots Minimakss algoritms, tas izmanto divus mainīgos Alfa (garantē labāko rezultātu MAX spēlētājam) un Beta (garantē labāko rezultātu min spēlētājam). Ja algoritms sāk pārbaudīt zaru un redz, ka pretinieks (MIN) šajā zarā spēs panākt situāciju, kas ir sliktāka par jau atrastu citu alternatīvu alfa >= beta, algoritms šo zaru tālāk nemaz nepārbauda to apgriež. Tas atgriezīs tādu pašu rezultātu kā Minimakss tikai izskaitļo to daudz ātrāk.

Rezultātu analīze:

Pirmajos piecos gājienos, kad cilvēks izvēlējās optimālākos gājienus, mākslīgais intelekts uzvarēja tikai tad kad izvēlējās iet pirmais. Ļoti vērtīgi ir pieminēt, ka spēlējot ar Alfa-Beta algoritmu datoram, pat sanāca gandrīz uzvarēt kad cilvēks gāja pirmais, bija neizšķirts un tas apliecina to faktu, ka Alfa-Beta algoritms, strādā efektīvāk par Minimakss.

Apkopotie dati apliecina, ka Alfa-Beta algoritms darbojas ievērojami efektīvāk nekā Minimakss algoritms. Tas patērē mazāk resursu, jo izvērtē mazāku virsotņu skaitu, kā arī nodrošina īsāku vidējo domāšanas laiku un stabilāku gājienu izpildi. Saskaņā ar aprēķiniem Alfa-Beta algoritms šajā gadījumā darbojas gandrīz 76 reizes ātrāk nekā Minimakss.

Vidējā Minimakss algoritma spēle - 0.718626s

Vidējā Alfa-Beta algoritma spēle - 0.009486s

Secinājumi:

Mūsu komanda saņēma sākotnējo uzdevumu - spēli, kas bija jāievieš. Spēles nosacījumi palika nemainīgi. Uzdevums ietvēra izvēli, kurš pirmais sāks spēli (cilvēks vai dators), izvēli, kuru algoritmu dators izmantos spēlē, spēles laukuma pielāgošanu un pašreizējās spēles restartēšanu pēc pabeigšanas.

Darba laikā mēs pielietojām un pētījām spēļu koka meklēšanas algoritmus un pielietojām tos divu spēlētāju spēlei, piemēram, Minimakss un tās uzlabotajai versijai Alfa-Beta.

Minimakss algoritma būtība ir divu spēlētāju iespējamo gājienu alternatīva uzskaitīšana, kurā mēs pieņemam, ka spēlētājs, "kura gājiens ir", izvēlēsies gājienu, kas dod maksimālo punktu skaitu. Minimakss pieņem, ka pretinieks vienmēr spēlē vislabāk, tādējādi iesakot optimālo gajēnu. Savukārt Alpha-Beta algoritms darbojas pēc tā paša principa, bet tā galvenais mērķis ir samazināt skaitļošanas laiku, apgriežot koka zarus, kas neietekmē gala rezultātu. Šī projekta spēļu koks tika ieviests, izmantojot TreeNode klasi, kur katrs mezgls saglabā noteiktu spēles stāvokli, veiktos gājienus un nākamos iespējamos stāvokļus. Šī struktūra ļauj meklēt dažādas gājiena iespējas un izvēlēties optimālo.

Spēles stāvokļa informācija tiek glabāta SolverState. Datiem tiek izmantoti korteži, kas palīdz saglabāt stāvokli nemainīgu un uzturēt to uzticamāk. Savukārt saraksti tiek izmantoti, ja dati ir jāmaina, piemēram, lai izveidotu jaunu spēles stāvokli.

Lai vienkāršotu programmas struktūru, tika izmantotas tādas datu klases kā Action, SolverState, SearchStats un TreeNode, lai skaidri organizētu programmas loģiku un atdalītu spēles stāvokļa datus no algoritmiskajiem aprēķiniem. Vārdnīcas tika izmantotas arī, lai sasaistītu dažādas programmas funkcijas un konfigurācijas parametrus.

Pētījuma laikā tika analizēta algoritma efektivitāte, atklājot, ka Alfa-Beta algoritms ir ievērojami efektīvāks, pateicoties zemākam skaitļošanas resursu patēriņam un nepieciešamībai analizēt mazāk mezglu. Kopumā var secināt, ka Alfa-Beta algoritms ir ievērojami efektīvāks nekā klasiskais Minimakss algoritms, jo tas ļauj analizēt mazāku virsotņu skaitu un ātrāk pieņemt lēmumus, vienlaikus saglabājot to pašu optimālo rezultātu.

Izmantotie informācijas avoti

https://www.geeksforgeeks.org/artificial-intelligence/mini-max-algorithm-in-artificial-intelligence/

https://www.geeksforgeeks.org/artificial-intelligence/alpha-beta-pruning-in-adversarial-search-algorithms/

https://www.geeksforgeeks.org/artificial-intelligence/mini-max-algorithm-in-artificial-intelligence/

https://www.geeksforgeeks.org/artificial-intelligence/heuristic-function-in-ai/

https://github.com/BVLC/caffe/wiki/Caffe-Output:-.caffemodel-.solverstate

https://www.hackerearth.com/blog/minimax-algorithm-alpha-beta-pruning

https://doc.qt.io/qt-6/reference-overview.html

https://doc.qt.io/qtforpython-6/

1. Pielikums(kods)

```
from __future__ import annotations

import math
import random
import sys
from time import perf_counter
from dataclasses import dataclass, field
from typing import Any, Callable, Dict, List, Literal, Optional

from PyQt6.QtCore import QSize, QTimer
from PyQt6.QtWidgets import QApplication, QGridLayout, QMessageBox, QWidget

from User_Input_Widget import UserInputWidget
from Game_Data_Widget import GameDataWidget
from AI_Config_Widget import AIcfgWidget
from Game_State import GameState


@dataclass(frozen=True)
class Action:
    action_type: Literal["take", "split2", "split4"]
    index: int


@dataclass(frozen=True)
class SolverState:
    sequence: tuple[int, ...]
    scores: tuple[int, int]
    current_player: int

    def is_terminal(self) -> bool:
        return len(self.sequence) == 0


@dataclass
class TreeNode:
    state: SolverState
    action: Optional[Action] = None
    children: list["TreeNode"] = field(default_factory=list)
    value: Optional[float] = None
    depth: int = 0
```

```python
@dataclass
class SearchStats:
    generated_nodes: int = 0
    evaluated_nodes: int = 0
    total_move_time: float = 0.0
    move_count: int = 0

    def reset_search_stats(self) -> None:
        self.generated_nodes = 0
        self.evaluated_nodes = 0
        self.last_move_time = 0.0

    @property
    def average_move_time(self) -> float:
        if self.move_count == 0:
            return 0.0
        return self.total_move_time / self.move_count


class NumberSequenceGame:
    def __init__(self, max_depth: int = 4):
        self.max_depth = max_depth
        self.stats = SearchStats()
        self.last_tree_root: Optional[TreeNode] = None

    def initial_state(self, baseLength: int, starting_player: int) -> SolverState:
        sequence = tuple(random.randint(1, 4) for _ in range(baseLength))
        return SolverState(
            sequence=sequence,
            scores=(0, 0),
            current_player=starting_player
        )

    def reset_search_stats(self) -> None:
        self.stats.reset_search_stats()
        self.last_tree_root = None

    def get_legal_actions(self, state: SolverState) -> List[Action]:
        actions: List[Action] = []

        for i, value in enumerate(state.sequence):
            actions.append(Action("take", i))

            if value == 2:
                actions.append(Action("split2", i))
            elif value == 4:
                actions.append(Action("split4", i))
```

```
        return actions

    def apply_action(self, state: SolverState, action: Action) -> SolverState:
        seq = list(state.sequence)
        scores = list(state.scores)
        player = state.current_player

        if action.index < 0 or action.index >= len(seq):
            raise ValueError("Nederīgs indekss.")

        value = seq[action.index]

        if action.action_type == "take":
            scores[player] += value
            del seq[action.index]

        elif action.action_type == "split2":
            if value != 2:
                raise ValueError('"split2" drīkst izmantot tikai skaitlim 2.')
            seq[action.index:action.index + 1] = [1, 1]

        elif action.action_type == "split4":
            if value != 4:
                raise ValueError('"split4" drīkst izmantot tikai skaitlim 4.')
            scores[player] += 1
            seq[action.index:action.index + 1] = [2, 2]

        else:
            raise ValueError("Nezināms darbības tips.")

        return SolverState(
            sequence=tuple(seq),
            scores=(scores[0], scores[1]),
            current_player=1 - player
        )

    def evaluate_terminal(self, state: SolverState, maximizing_player: int) -> int:
        return state.scores[maximizing_player] - state.scores[1 - maximizing_player]

    def heuristic(self, state: SolverState, maximizing_player: int) -> float:
        my_score = state.scores[maximizing_player]
        opp_score = state.scores[1 - maximizing_player]
        score_diff = my_score - opp_score

        seq = state.sequence
        count1 = seq.count(1)
        count2 = seq.count(2)
        count3 = seq.count(3)
        count4 = seq.count(4)
```

```
        material = count1 * 1.0 + count2 * 2.0 + count3 * 3.0 + count4 * 4.0
        split_potential = count4 * 0.6 + count2 * 0.2
        turn_bonus = 0.3 if state.current_player == maximizing_player else -0.3

        return score_diff * 10 + material * 0.15 + split_potential + turn_bonus

    def order_actions(self, state: SolverState, actions: List[Action]) -> List[Action]:
        def priority(action: Action) -> int:
            value = state.sequence[action.index]

            if action.action_type == "take":
                return 100 + value
            if action.action_type == "split4":
                return 80
            if action.action_type == "split2":
                return 50
            return 0

        return sorted(actions, key=priority, reverse=True)

    def build_tree(
        self,
        state: SolverState,
        depth: int,
        maximizing_player: Optional[int] = None
    ) -> TreeNode:

        if maximizing_player is None:
            maximizing_player = state.current_player

        self.reset_search_stats()
        root = self._build_tree_recursive(state, depth, maximizing_player)
        self.last_tree_root = root
        return root

    def _build_tree_recursive(
        self,
        state: SolverState,
        depth: int,
        maximizing_player: int,
        action: Optional[Action] = None
    ) -> TreeNode:
        self.stats.generated_nodes += 1
        node = TreeNode(state=state, action=action, depth=depth)

        if state.is_terminal():
            self.stats.evaluated_nodes += 1
            node.value = self.evaluate_terminal(state, maximizing_player)
            return node
```

```
        if depth == 0:
            self.stats.evaluated_nodes += 1
            node.value = self.heuristic(state, maximizing_player)
            return node

        actions = self.order_actions(state, self.get_legal_actions(state))
        child_values: list[float] = []

        for next_action in actions:
            next_state = self.apply_action(state, next_action)
            child = self._build_tree_recursive(
                next_state,
                depth - 1,
                maximizing_player,
                next_action
            )
            node.children.append(child)
            if child.value is not None:
                child_values.append(child.value)

        if child_values:
            if state.current_player == maximizing_player:
                node.value = max(child_values)
            else:
                node.value = min(child_values)

        return node

    def minimax(
        self,
        state: SolverState,
        depth: int,
        maximizing_player: int,
        node: Optional[TreeNode] = None
    ) -> tuple[float, Optional[Action]]:
        if node is None:
            node = TreeNode(state=state, depth=depth)
            self.stats.generated_nodes += 1
            self.last_tree_root = node

        if state.is_terminal():
            self.stats.evaluated_nodes += 1
            value = self.evaluate_terminal(state, maximizing_player)
            node.value = value
            return value, None

        if depth == 0:
            self.stats.evaluated_nodes += 1
            value = self.heuristic(state, maximizing_player)
            node.value = value
```

```
        return value, None

    actions = self.order_actions(state, self.get_legal_actions(state))
    is_max_turn = state.current_player == maximizing_player
    best_action: Optional[Action] = None

    if is_max_turn:
        best_value = -math.inf

        for action in actions:
            next_state = self.apply_action(state, action)
            child = TreeNode(state=next_state, action=action, depth=depth - 1)
            self.stats.generated_nodes += 1
            node.children.append(child)

            value, _ = self.minimax(
                next_state,
                depth - 1,
                maximizing_player,
                child
            )

            if value > best_value:
                best_value = value
                best_action = action

        node.value = best_value
        return best_value, best_action

    best_value = math.inf

    for action in actions:
        next_state = self.apply_action(state, action)
        child = TreeNode(state=next_state, action=action, depth=depth - 1)
        self.stats.generated_nodes += 1
        node.children.append(child)

        value, _ = self.minimax(
            next_state,
            depth - 1,
            maximizing_player,
            child
        )

        if value < best_value:
            best_value = value
            best_action = action

    node.value = best_value
    return best_value, best_action
```

```python
    def alphabeta(
        self,
        state: SolverState,
        depth: int,
        alpha: float,
        beta: float,
        maximizing_player: int,
        node: Optional[TreeNode] = None
    ) -> tuple[float, Optional[Action]]:
        if node is None:
            node = TreeNode(state=state, depth=depth)
            self.stats.generated_nodes += 1
            self.last_tree_root = node

        if state.is_terminal():
            self.stats.evaluated_nodes += 1
            value = self.evaluate_terminal(state, maximizing_player)
            node.value = value
            return value, None

        if depth == 0:
            self.stats.evaluated_nodes += 1
            value = self.heuristic(state, maximizing_player)
            node.value = value
            return value, None

        actions = self.order_actions(state, self.get_legal_actions(state))
        is_max_turn = state.current_player == maximizing_player
        best_action: Optional[Action] = None

        if is_max_turn:
            value = -math.inf

            for action in actions:
                next_state = self.apply_action(state, action)
                child = TreeNode(state=next_state, action=action, depth=depth - 1)
                self.stats.generated_nodes += 1
                node.children.append(child)

                child_value, _ = self.alphabeta(
                    next_state,
                    depth - 1,
                    alpha,
                    beta,
                    maximizing_player,
                    child
                )

                if child_value > value:
```

```
                value = child_value
                best_action = action

            alpha = max(alpha, value)
            if beta <= alpha:
                break

        node.value = value
        return value, best_action

    value = math.inf

    for action in actions:
        next_state = self.apply_action(state, action)
        child = TreeNode(state=next_state, action=action, depth=depth - 1)
        self.stats.generated_nodes += 1
        node.children.append(child)

        child_value, _ = self.alphabeta(
            next_state,
            depth - 1,
            alpha,
            beta,
            maximizing_player,
            child
        )

        if child_value < value:
            value = child_value
            best_action = action

        beta = min(beta, value)
        if beta <= alpha:
            break

    node.value = value
    return value, best_action

def best_move(self, state: SolverState, algorithm: str = "alphabeta") -> Optional[Action]:
    maximizing_player = state.current_player
    algorithm = algorithm.lower()

    self.reset_search_stats()

    start = perf_counter()

    if algorithm == "minmax":
        _, action = self.minimax(
            state,
            depth=4,
```

```
            maximizing_player=maximizing_player
        )
    elif algorithm == "alphabeta":
        _, action = self.alphabeta(
            state,
            4,
            -math.inf,
            math.inf,
            maximizing_player
        )
    else:
        raise ValueError("Algoritmam jābūt 'minmax' vai 'alphabeta'.")

    elapsed = perf_counter() - start
    self.stats.last_move_time = elapsed
    self.stats.total_move_time += elapsed
    self.stats.move_count += 1

    return action

  def get_experiment_data(self) -> dict[str, Any]:
    return {
        "generated_nodes": self.stats.generated_nodes,
        "evaluated_nodes": self.stats.evaluated_nodes,
        "total_move_time": self.stats.total_move_time,
        "average_move_time": self.stats.average_move_time,
        "last_move_time": self.stats.last_move_time,
        "move_count": self.stats.move_count,
        "tree_root": self.last_tree_root,
    }


class GameDispatcher:
  def __init__(self):
    self.baseLength: int = 15

    self.isActive: bool = False
    self.activeStatusDependedFunctions: Dict[
        Callable[..., Any],
        Literal["SendValue", "Execute"]
    ] = {}

    self.algorithm: Literal["minmax", "alphabeta"] | None = None
    self.algorithmDependedFunctions: Dict[
        Callable[..., Any],
        Literal["SendValue", "Execute"]
    ] = {}

    self.engine = NumberSequenceGame(max_depth=4)
    self.state = GameState()
```

```
        self.window: Optional[QWidget] = None
        self.startingPlayer: Literal[0, 1] = 0

        self.humanWins: int = 0
        self.aiWins: int = 0

        self.game_generated_nodes: int = 0
        self.game_evaluated_nodes: int = 0
        self.game_ai_total_time: float = 0.0
        self.game_ai_move_count: int = 0

    def setStartingPlayer(self, value: Literal[0, 1]) -> None:
        self.startingPlayer = value

    def set_window(self, window: QWidget) -> None:
        self.window = window

    def algorithmChangeSubscriber(
        self,
        function: Callable[..., Any],
        activationRule: Literal["SendValue", "Execute"]
    ) -> None:
        self.algorithmDependedFunctions[function] = activationRule

    def gameStateChangeSubscriber(
        self,
        function: Callable[..., Any],
        activationRule: Literal["SendValue", "Execute"]
    ) -> None:
        self.activeStatusDependedFunctions[function] = activationRule

    def _notifyAlgorithmChange(self) -> None:
        for function, rule in self.algorithmDependedFunctions.items():
            function(self.algorithm) if rule == "SendValue" else function()

    def _notifyGameStateChange(self) -> None:
        for function, rule in self.activeStatusDependedFunctions.items():
            function(self.isActive) if rule == "SendValue" else function()

    def setNumberRowBaseLength(self, newLength: int) -> None:
        if not self.isActive:
            self.baseLength = newLength

    def getNumberRowBaseLength(self) -> int:
        return self.baseLength

    def _solverToView(self, solver_state: SolverState) -> GameState:
        new_state = GameState(
            sequence=solver_state.sequence,
            user_score=solver_state.scores[0],
```

```
            ai_score=solver_state.scores[1],
            user_turn=(solver_state.current_player == 0),
            algorithm=self.algorithm,
            is_finished=(len(solver_state.sequence) == 0),
        )
        new_state.userWins = self.state.userWins
        new_state.aiWins = self.state.aiWins
        return new_state

    def _viewToSolver(self) -> SolverState:
        return SolverState(
            sequence=self.state.sequence,
            scores=(self.state.user_score, self.state.ai_score),
            current_player=0 if self.state.user_turn else 1,
        )

    def _finishGame(self) -> None:
        self.state.is_finished = True

        if self.state.user_score > self.state.ai_score:
            self.state.incrementHumanWins()
            self.state.winner_text = (
                f"Spēle beigusies. Jūs uzvarējāt, rezultāts ir "
                f"{self.state.user_score}:{self.state.ai_score}."
            )
        elif self.state.ai_score > self.state.user_score:
            self.state.incrementAiWins()
            self.state.winner_text = (
                f"Spēle beigusies. Uzvarēja MI, rezultāts ir "
                f"{self.state.user_score}:{self.state.ai_score}."
            )
        else:
            self.state.winner_text = (
                f"Spēle beigusies neizšķirti. Rezultāts ir "
                f"{self.state.user_score}:{self.state.ai_score}."
            )

        self.isActive = False
        self._notifyGameStateChange()
        avg_ai_time = (
        self.game_ai_total_time / self.game_ai_move_count
        if self.game_ai_move_count > 0 else 0.0
        )

        print("\n===== SPELES REZULTATI =====")
        print(f"Kopejais genereto virsotnu skaits: {self.game_generated_nodes}")
        print(f"Kopejais noverteto virsotnu skaits: {self.game_evaluated_nodes}")
        print(f"MI videjais laiks gajienam: {avg_ai_time:.6f}s")
        print(f"Sakuma virknes garums: {self.baseLength}")
        print(f"Algoritms: {self.algorithm}")
```

```
        print("============================\n")
        if self.window is not None:
            QMessageBox.information(self.window, "Spēle beidzās", self.state.winner_text)

    def _applySolverAction(self, action: Action) -> None:
        next_state = self.engine.apply_action(self._viewToSolver(), action)
        self.state = self._solverToView(next_state)

        if self.state.is_finished:
            self._finishGame()
        else:
            self._notifyGameStateChange()

    def _findFirstValueIndex(self, value: int) -> int:
        try:
            return self.state.sequence.index(value)
        except ValueError as exc:
            raise ValueError(f"Skaitlis {value} virknē nav pieejams.") from exc

    def doUserTurn(
        self,
        action: Literal["TAKE1", "TAKE2", "TAKE3", "TAKE4", "SPLIT2", "SPLIT4"]
    ) -> None:
        if not self.isActive or not self.state.user_turn or self.state.is_finished:
            return

        try:
            if action == "TAKE1":
                selected_action = Action("take", self._findFirstValueIndex(1))
            elif action == "TAKE2":
                selected_action = Action("take", self._findFirstValueIndex(2))
            elif action == "TAKE3":
                selected_action = Action("take", self._findFirstValueIndex(3))
            elif action == "TAKE4":
                selected_action = Action("take", self._findFirstValueIndex(4))
            elif action == "SPLIT2":
                selected_action = Action("split2", self._findFirstValueIndex(2))
            elif action == "SPLIT4":
                selected_action = Action("split4", self._findFirstValueIndex(4))
            else:
                return

            self._applySolverAction(selected_action)

        except ValueError:
            return

        if self.isActive and not self.state.user_turn and not self.state.is_finished:
            QTimer.singleShot(200, self._doAiTurn)
```

```
    def _doAiTurn(self) -> None:
        if not self.isActive or self.state.user_turn or self.state.is_finished:
            return

        solver_state = self._viewToSolver()
        ai_action = self.engine.best_move(solver_state, self.algorithm or "alphabeta")

        data = self.engine.get_experiment_data()

        self.game_generated_nodes += data["generated_nodes"]
        self.game_evaluated_nodes += data["evaluated_nodes"]
        self.game_ai_total_time += data["last_move_time"]
        self.game_ai_move_count += 1

        if ai_action is None:
            self._finishGame()
            return

        self._applySolverAction(ai_action)

    def doConfigurateAi(self, action: Literal["minmax", "alphabeta"]) -> None:
        self.algorithm = action
        self._notifyAlgorithmChange()

    def getGameState(self) -> GameState:
        return self.state

    def getExperimentData(self) -> dict[str, Any]:
        return self.engine.get_experiment_data()

    def buildExperimentTree(self, depth: int | None = None) -> Optional[TreeNode]:
        if self.state.is_finished:
            return None

        solver_state = self._viewToSolver()
        root = self.engine.build_tree(
            solver_state,
            depth if depth is not None else self.engine.max_depth,
            solver_state.current_player
        )
        return root

    def changeActiveState(self) -> None:
        if self.algorithm is None:
            return

        self.game_generated_nodes = 0
        self.game_evaluated_nodes = 0
        self.game_ai_total_time = 0.0
        self.game_ai_move_count = 0
```

```
        prev_user_wins = self.state.userWins
        prev_ai_wins = self.state.aiWins

        self.state = self._solverToView(
            self.engine.initial_state(self.baseLength, starting_player=self.startingPlayer)
        )

        self.state.userWins = prev_user_wins
        self.state.aiWins = prev_ai_wins

        self.state.algorithm = self.algorithm
        self.state.winner_text = ""
        self.state.is_finished = False
        self.isActive = True
        self._notifyGameStateChange()

        if not self.state.user_turn:
            QTimer.singleShot(200, self._doAiTurn)


GAME_DISPATCHER = GameDispatcher()


class MainWindow(QWidget):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("MIP-1")
        self.setMinimumSize(QSize(500, 400))

        self.layout = QGridLayout()
        self.setLayout(self.layout)

        GAME_DISPATCHER.set_window(self)

        self.gameDataWidget = GameDataWidget(
            GAME_DISPATCHER.getGameState,
            GAME_DISPATCHER.changeActiveState,
            GAME_DISPATCHER.algorithmChangeSubscriber,
            GAME_DISPATCHER.gameStateChangeSubscriber,
            GAME_DISPATCHER.setNumberRowBaseLength,
            GAME_DISPATCHER.getNumberRowBaseLength
        )

        self.userInputWidget = UserInputWidget(
            GAME_DISPATCHER.doUserTurn,
            GAME_DISPATCHER.getGameState().getTurn,
            GAME_DISPATCHER.gameStateChangeSubscriber,
            GAME_DISPATCHER.getGameState
```

```
        )

        self.aiIoWidget = AIcfgWidget(
            GAME_DISPATCHER.doConfigurateAi,
            GAME_DISPATCHER.gameStateChangeSubscriber,
            GAME_DISPATCHER.setStartingPlayer
        )

        self.layout.addWidget(self.gameDataWidget, 0, 0, 1, 2)
        self.layout.addWidget(self.userInputWidget, 1, 0, 1, 1)
        self.layout.addWidget(self.aiIoWidget, 1, 1, 1, 1)

def main():
    app = QApplication(sys.argv)
    gameWindow = MainWindow()
    gameWindow.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
else:
    del GAME_DISPATCHER
```

2. Pielikums

Ar zaļo krāsu tiek atzīmēts, kur ir izmantots mākslīgais intelekts.

https://chatgpt.com/share/69c2833e-90b8-800c-ab54-1b02e7842a84

https://chatgpt.com/share/69c29a1e-0bf8-800c-ab61-26f2f21eca25

https://chatgpt.com/share/69c412cb-8324-8391-91bf-12c45f4f0759

https://chatgpt.com/share/69c41584-58cc-8396-a674-4f5473f77772

https://gemini.google.com/app/2b28913f9b700941